

AI-Assisted Black Box Testing 2

Dr Chris Meudec 2026-2038

Black Box Test Cases Generation

- We have to examine the input domain (the universe of possible inputs) given the intended behaviour of the program under test, and then select inputs that have a high probability of revealing an error
- Typically, this process can be based on :
 - Experience, guesswork or intuition of the test engineer;
 - A random choice;
 - Equivalence partitioning;
 - Boundary value analysis;
 - A combination of all of the above.

Black Box Test Cases Generation

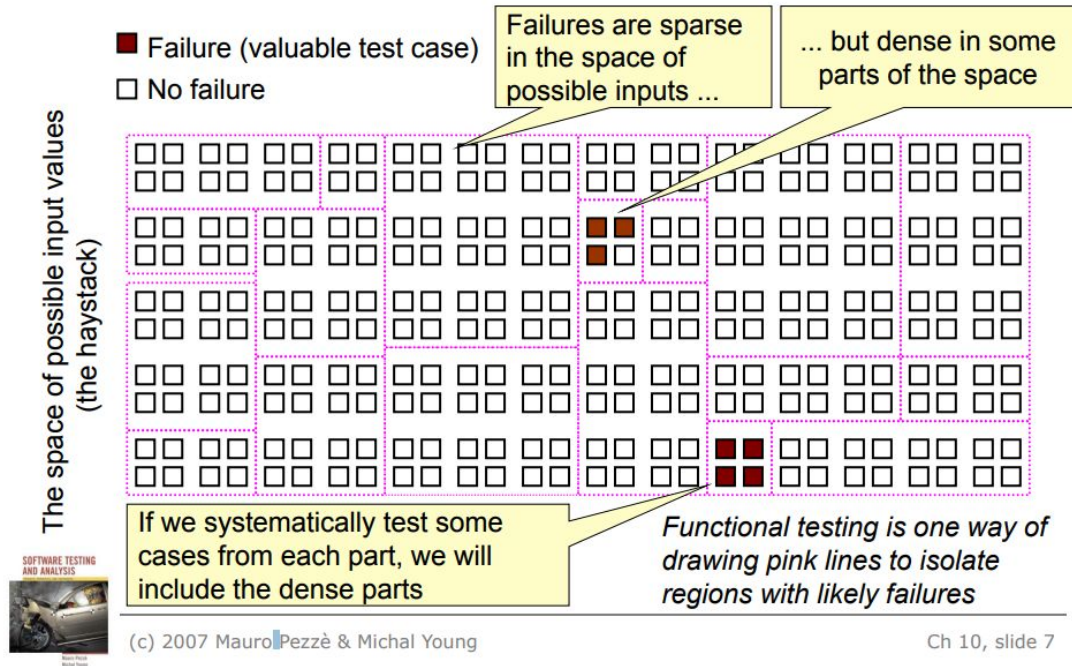
- Divides the input domain into different equivalence classes
 - Implies partitioning the output domain too...
- An **equivalence class** is a chunk of the input domain for which the **behaviour of the program is somehow the same** (i.e. programs normally behave in a comparable way for all inputs of an equivalence partition).
 - This is an assumption: may be right, may be wrong
 - The idea is that it is sufficient to test one member of a partition to reveal the behaviour of the program for the entire partition
 - Allows to reduce the number of necessary test cases

Robustness Testing

- To ensure the system, method, API, CLI is robust i.e. does not accept unexpected inputs we have to test using unexpected input values and ensure that the system's behaviour is adequate, e.g.:
 - ignore the request completely and do nothing
 - log the unexpected occurrence
 - issue an error and stop or a warning and continue
- So invalid input domains must also be partitioned for system that must be robust

What is Equivalence Partitioning?

Systematic Partition Testing



Equivalence Partitioning

- A test case must be created for each equivalence class
 - The test cases must be run against the code using a tool to verify the system's behaviour is as expected
 - The more complete the oracle the better
 - a failed test may indicate:
 - » a mistake in the code of the automated test
 - » the oracle is wrong
 - » the system's behaviour is wrong
 - » all of the above
 - » => require a thorough investigation

Combining with White-Box Testing

- Code coverage analysis can be performed by testing tools to ensure that the code has been thoroughly exercised
 - if the coverage (e.g. branch coverage) achieved after Black Box Testing, i.e. if coverage analysis shows unexercised code, what do you do?
 - » ask yourself why
 - » because ???
 - » what are you gonna do about it?
 - » One good answer is that unexercised code is behaviour the partitions didn't anticipate. That is exactly where unexpected outputs live...

Equivalence Partitioning Example

A revealing, basic example: calculate your final grade

- Separate Document: [Black Box Test Cases Generation Examples](#)

Equivalence Partitioning Advice

1. Use a spreadsheet, be systematic
2. Identify all the inputs
3. for each input identify the valid partitions and the invalid partitions
4. Identify the outputs (the "behaviour")
5. for each output / behaviour identify the valid partitions and *possible invalid output* (that's speculative)
6. for each partition, pick the inputs and create a test case with the expected behaviour

Exercise A, disk quota

It is a quota system for disk usage:

- If the student is in first year the quota should always be 20Mb;
- If the student is in second year the quota should be 30MB;
- If the student is in third year and in computing the quota should be 60MB;
- If the student is in third year and not in computing the quota should be 40MB;
- If the student is in fourth year and in computing the quota should be 100MB;
- If the student is in fourth year and in physics the quota should be 80MB;
- If the student is in fourth year and in not in computing nor physics the quota should be 60MB;

In pairs, derive robust test cases for the system as described above use Google Sheets, put your answers in the class shared folder (available in the module's page). Respect this layout:

- Tab Partitions. Format: ID, Input or output, Partition, Valid or invalid
- Tab Test Case. Format: ID, Year, Department, Expected quota, Partitions ID covered
- Tab Queries. Format: Query, Affects tests, Assumption made

Exercise B, LLM versus the nineteen

Enter the following in your favorite LLM (The same example as earlier):

Consider a component, `generate_grading`, with the following specification: The component is passed an exam mark (out of 75) and a coursework (c/w) mark (out of 25), from which it generates a grade for the course in the range 'A' to 'D'. The grade is calculated from the overall mark which is calculated as the sum of the exam and c/w marks, as follows: greater than or equal to 70 – 'A'; greater than or equal to 50, but less than 70 – 'B'; greater than or equal to 30, but less than 50 – 'C'; less than 30 – 'D'. Where a mark is outside its expected range then a fault message ('FM') is generated. All inputs are passed as integers. Identify all the equivalence partitions for this component, including input and output partitions, valid and invalid and output the partitions in a table in using this format: ID, Input or output, Partition, Valid or invalid.

Assess the quality of the partitions versus the 19 that were generated manually:

- how many partitions are generated?
- what is missing?
- Are the partitions always clear?
- Is there some confusing about what is a valid and invalid?

Exercise B, LLM versus the nineteen

- Further ask:

What outputs might this component produce that the specification does not mention, and what implementation faults would cause them?

- The takeaway:

- the LLMs partition what the specification mentions.
- It doesn't ask, unprompted, what the component might do that the specification never mentions.
- Asking that is the human tester's job.